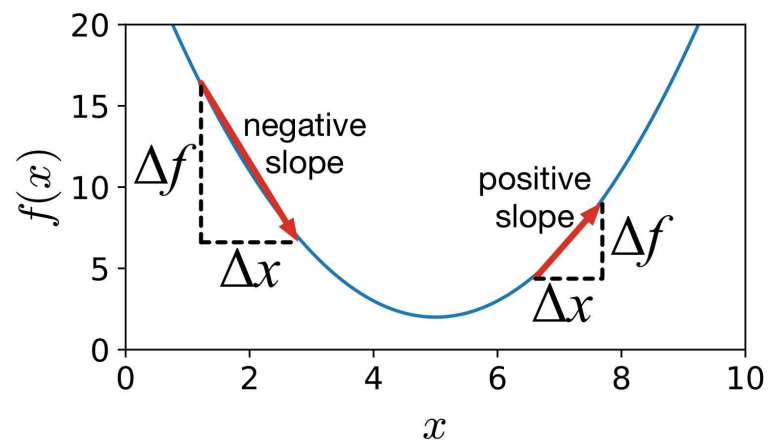
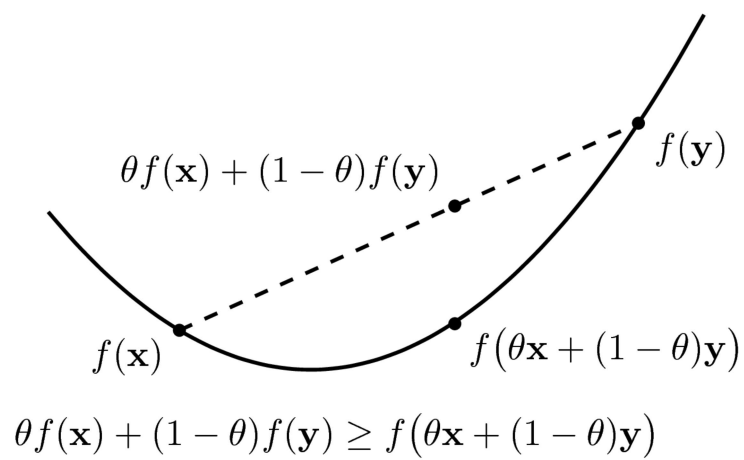


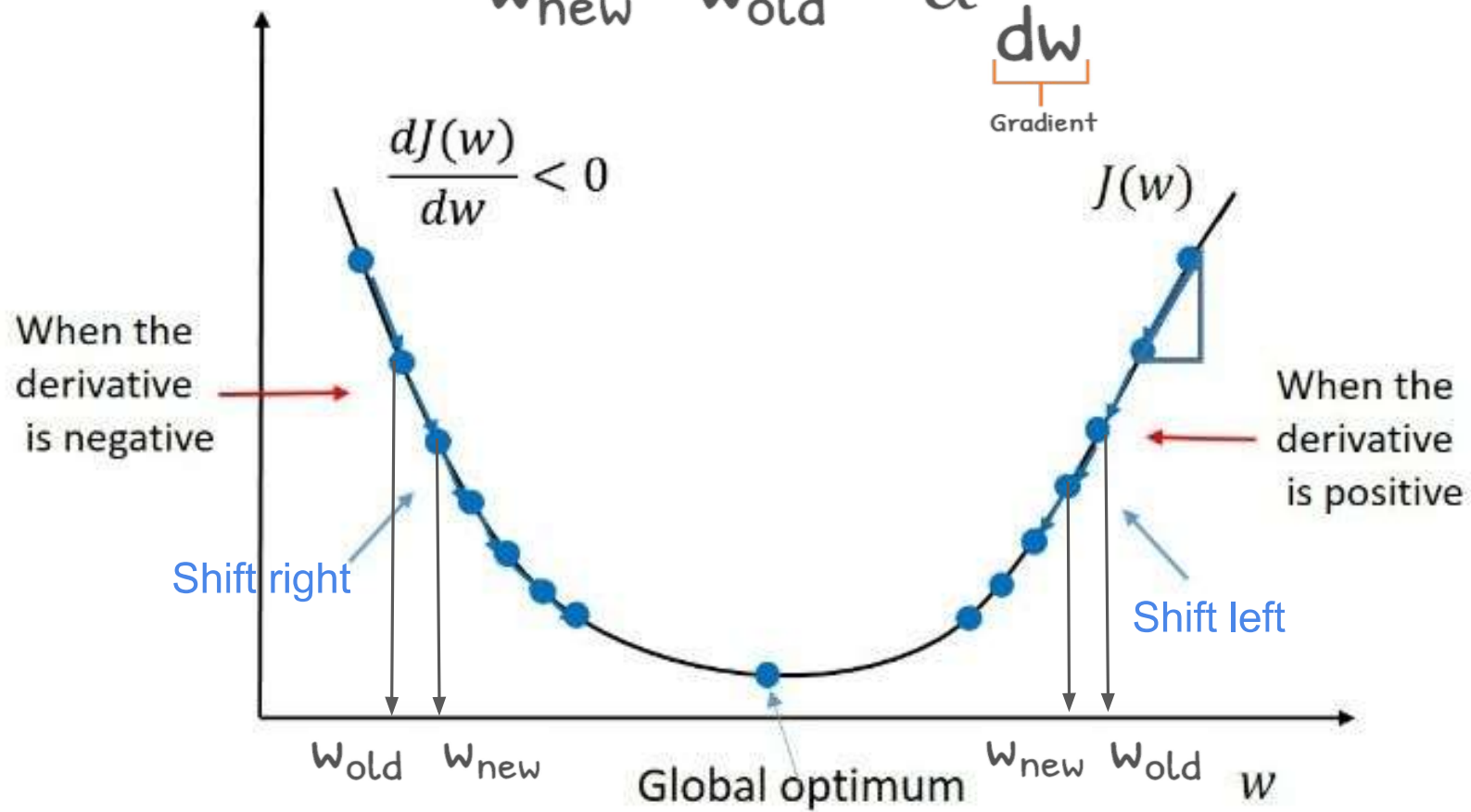
Deep Learning

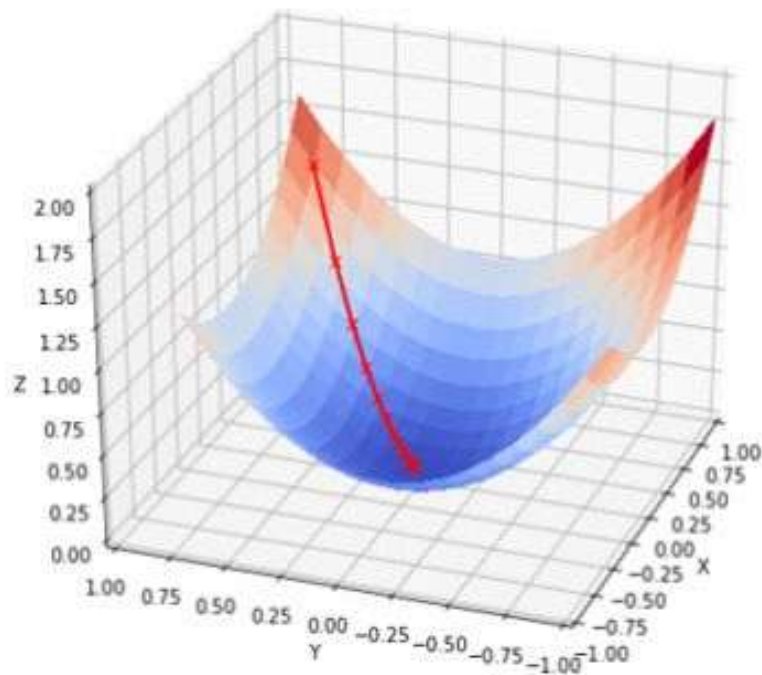
lec-2

Gradient descent

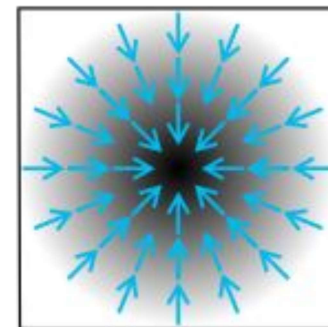
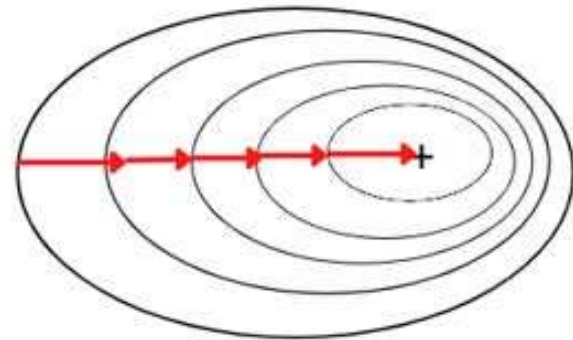


$$w_{\text{new}} = w_{\text{old}} - \alpha \underbrace{\frac{dJ}{dw}}_{\text{Gradient}}$$



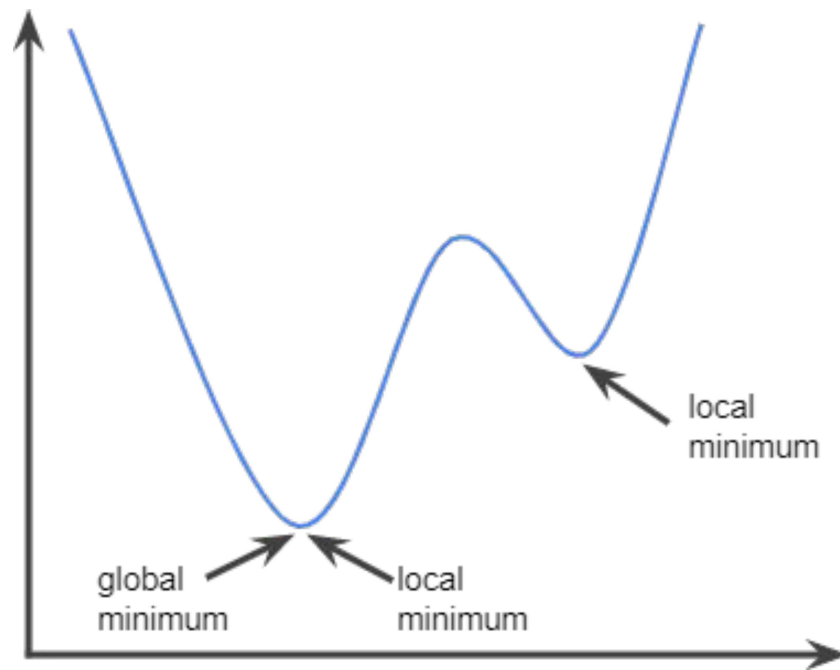


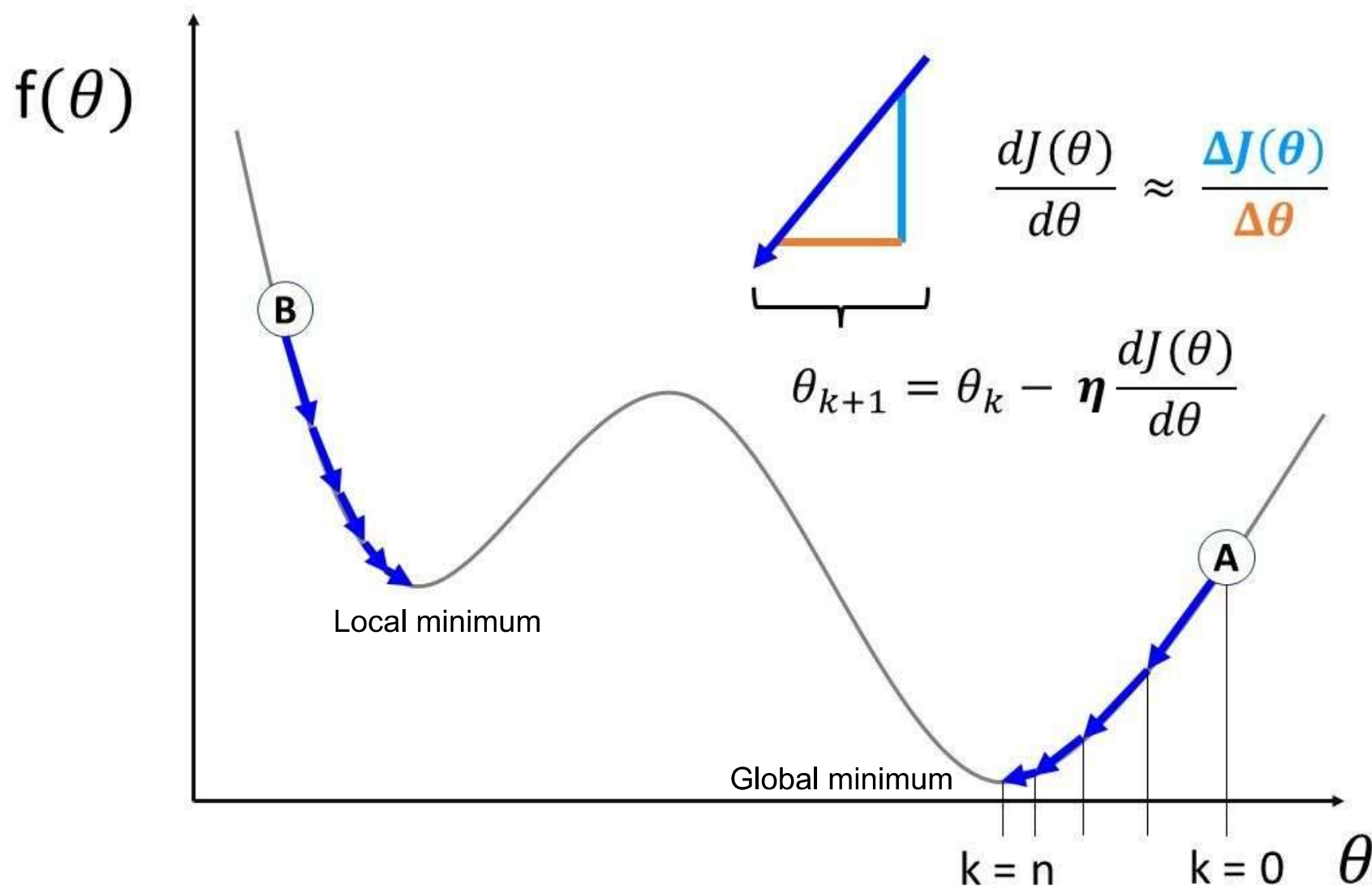
Contours



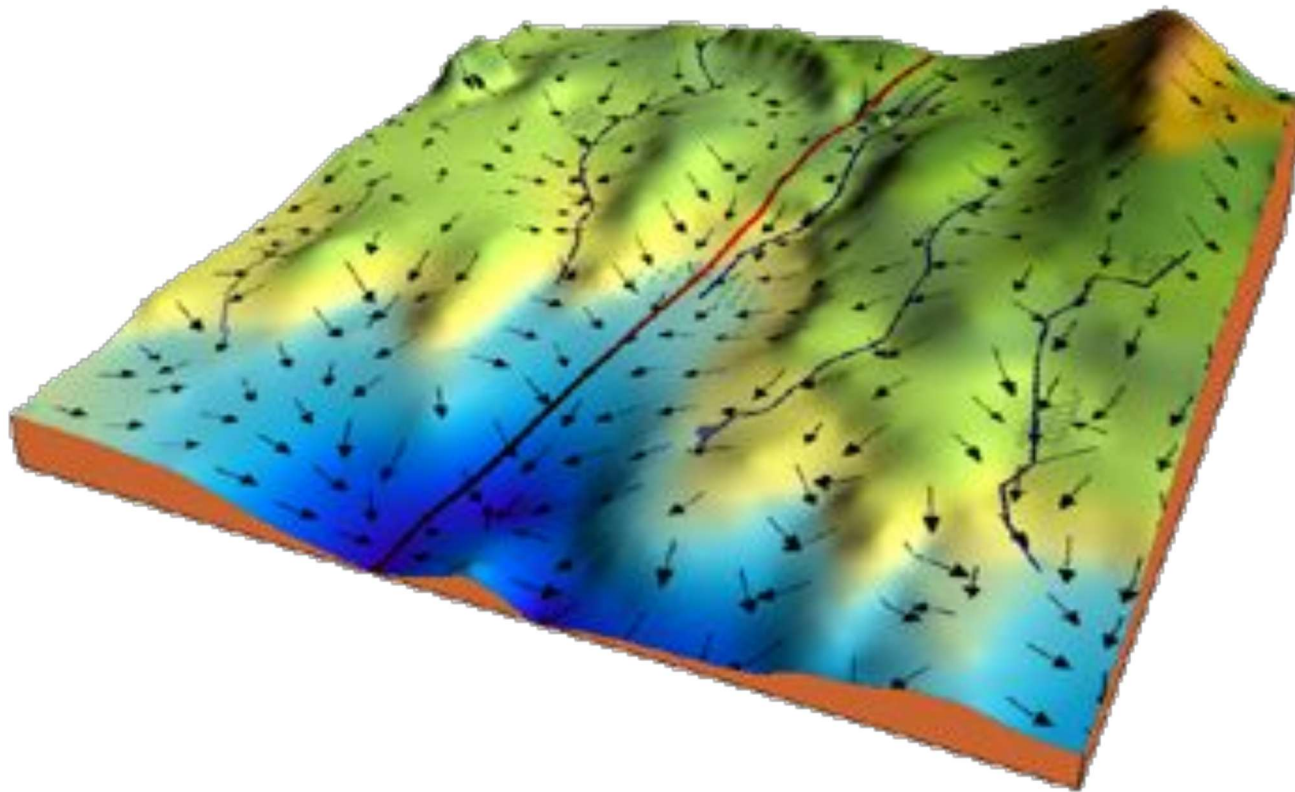
Gradient flow for a perfect convex surface

Non convex functions





In general objective is to achieve Steepest descent point overcoming local minima:
Optimizers



Feed Forward Neural Network:

<https://drive.google.com/file/d/1OJhGgrjIG30IzpCLvuwx8HuJ5hP3dkC5/view?usp=sharing>

Item	Feed-Forward NN (BP)
Forward pass	$h = f(W^{(1)}x)$ $y = g(W^{(2)}h)$
Loss	$L = \ell(y, \hat{y})$
Output-layer error	$\delta^{(2)} = \frac{\partial L}{\partial a^{(2)}}$
Hidden-layer error	$\delta^{(1)} = (W^{(2)})^\top \delta^{(2)} \odot f'(a^{(1)})$
Weight update – output	$\Delta W^{(2)} = -\eta \delta^{(2)} h^\top$
Weight update – input/hidden	$\Delta W^{(1)} = -\eta \delta^{(1)} x^\top$